```
                type: string
                example: Movie not found
```

The expectations of the responses effectively become the mocks within frontend tests and become data used in backend test scenarios. As we've learned throughout the book so far, each part of the specification shown in listing 12.1 can be covered by a contract test. Implementing these scenarios as contract tests will catch any deviation from the implementation stated within the specification. In addition, contract tests provide earlier feedback before deploying changes to an environment. Getting early feedback saves teams a lot of time going back and forth. Once code has been committed and deployed to an environment, it's harder to debug, meaning it's slower to fix.

12.2 Identifying suitable integration tests to migrate

Lots of integration tests could be converted to contract tests. It's important to understand the separation of the purpose between the two types of tests. Suppose we look at the difference between unit tests and integration tests first. Unit tests run in isolation by mocking out any dependencies, whereas integration tests verify the actual dependencies. Now, if we compare unit tests with contract tests, we see that both use mocks. However, mocks created within contract tests are shared with provider services.

There are a few other distinctions to make when comparing integration and contract tests in relation to test capabilities. There will be some duplication between the tests around status codes and data structures. However, the level of detail and number of scenarios for each area will defer. Contract tests focus on verifying the specific details around the following:

- Data structure
- HTTP status codes
- Request and response headers
- API versions